Introduction to Digital Systems

Part II (3 lectures)

2025/2026

Combinational Logic Blocks

Arnaldo Oliveira, Augusto Silva, Iouliia Skliarova, Adão Silva

Lecture 7 contents

- Block oriented combinational logic design
- Arithmetic Circuits

Addition

- Addition is a very common arithmetic operation in digital systems
- An *adder* combines two arithmetic operands using the addition rules described before.
- Addition and subtraction of non-decimal numbers by hand uses the same technique that you know from school for decimal numbers.
- The only catch is that the addition and subtraction tables are different.

- Let´s recall some concepts …

Addition of Binary Numbers

- To add two **unsigned binary numbers** *X* and *Y*, we add together the least significant bits with an initial carry (c_{in}) of 0, producing carry (c_{out}) and sum (s) bits according to the table. We continue processing bits from right to left, adding the carry out of each column into the next column's sum.

Example:

$$
\begin{array}{rcccccccc}
 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & \\
 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\
+ & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\
\hline
 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0
\end{array}
$$

c_{in}	x	y	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Subtraction of Binary Numbers

- Binary subtraction is performed similarly, using borrows (b_{in} and b_{out}) instead of carries between steps, and producing a difference bit *d*.

Examples:

```
   0 1 1 1 1 0 0
   1 1 1 0 0 0 0 1
 - 1 0 1 0 1 1 0 1
 -----------------
   0 0 1 1 0 1 0 0
```

```
   1 1 1
   1 0 0 0
 - 0 0 1 1
 ---------
   0 1 0 1
```

b_{in}	x	y	b_{out}	d
0	0	0	0	0
0	0	1	1	1
0	1	0	0	1
0	1	1	0	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	1	1

Overflow

- With *n* bits it is possible to represent **unsigned integer numbers** ranging from 0 to 2^n-1.
- If an arithmetic operation produces a result that exceeds the range of the number system, **overflow** is said to occur.
- Overflows can easily be detected by analyzing **a carry or borrow from the most significant bit**.
  - the carry bit c_{out} or the borrow bit b_{out} out of the MSB = 1

Examples:

n=8: [0..255]

$173_{10} + 97_{10} = 270_{10}$

```
1 1 1 0 0 0 0 1
  1 0 1 0 1 1 0 1
+ 0 1 1 0 0 0 0 1
-----------------
  0 0 0 0 1 1 1 0
```

overflow

n=4: [0..15]

$4_{10} - 11_{10} = -7_{10}$

```
1 0 1 1
  0 1 0 0
- 1 0 1 1
---------
  1 0 0 1
```

overflow

Representation of Negative Numbers

- There are many ways to represent negative numbers.
- In everyday business we use the **signed-magnitude system** (i.e. reserve a special symbol to indicate whether a number is negative).

- However, most computers use **two's-complement representation**:
  - The **most significant bit** (**MSB**) of a number in this system serves as the sign bit; a number is negative if and only if its MSB is 1.
  - The weight of the MSB is negative: for an *n*-bit number the weight is -2^{n-1}.
  - The decimal equivalent for a two's-complement binary number is computed the same way as for an unsigned number, except that the weight of the MSB is negative:
    - D= $d_{n-1}d_{n-2} \dots d_1 d_0 = -2^{n-1} + \sum_{i=0}^{n-2} d_i \times 2^i$

Representation of Negative Numbers

Examples:

$1010_2 = ???_{10}$ $\quad$ $1010_2 = -2^3 + 2^1 = -8 + 2 = -6_{10}$

$1111_2 = ???_{10}$ $\quad$ $1111_2 = -2^3 + 2^2 + 2^1 + 2^0 = -8 + 4 + 2 + 1 = -1_{10}$

$0111_2 = ???_{10}$ $\quad$ $0111_2 = 2^2 + 2^1 + 2^0 = 4 + 2 + 1 = 7_{10}$

Two's Complement Representation

- For *n* bits, the range of representable numbers is $[-2^{n-1}, 2^{n-1}-1]$.

- For *n*=4, the range is [-8, 7]:

0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
-8	1	0	0	0
-7	1	0	0	1
-6	1	0	1	0
-5	1	0	1	1
-4	1	1	0	0
-3	1	1	0	1
-2	1	1	1	0
-1	1	1	1	1

Towards implementation

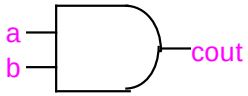
- The immediate approach
  - Digit-wise addition and carry propagation
  - Iterative hardware
  - Building blocks
    - Half-Adder
    - Full-Adder

The Half-Adder (HA)

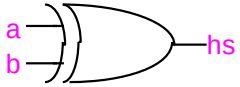
- Inputs: 2 single bit operands (a, b)
- Outputs
  - A 2 bit result:
    - The Half-Sum (hs)
    - The Carry-out (Cout)
    - Note that: $0 \leq (Cout, hs)_{10} \leq 2$

a	b	c_{out}	hs
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$c_{out} = a \cdot b$$



$$hs = a \oplus b$$



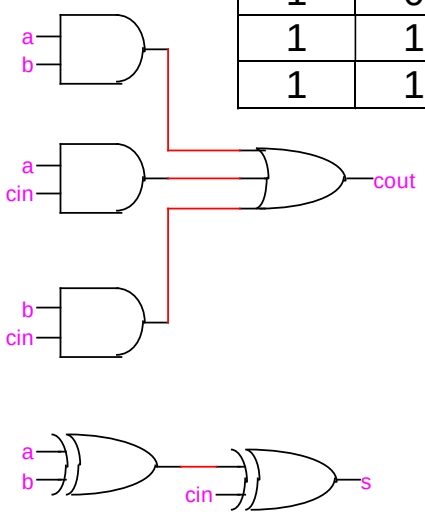
The Full-Adder (FA)

- To add operands with more than one bit, we must provide for carriers between bit positions.
- Inputs: 2 single bit operands (a, b) and a Carry-in bit (C_{in})
- Outputs
  - A 2 bit result:
    - The Sum (S)
    - The Carry-out (C_{out})
    - Note that: $0 \leq (C_{out}, S)_{10} \leq 3$

c_{in}	a	b	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

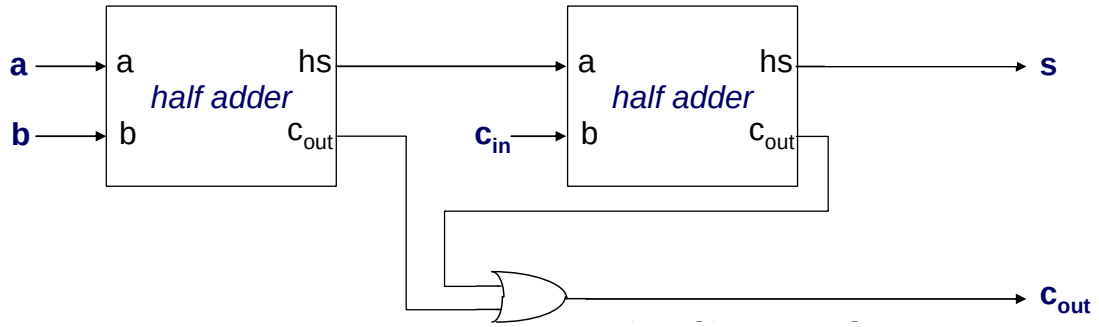
$$c_{out} = a \cdot b + a \cdot c_{in} + b \cdot c_{in}$$

$$s = a \oplus b \oplus c_{in}$$



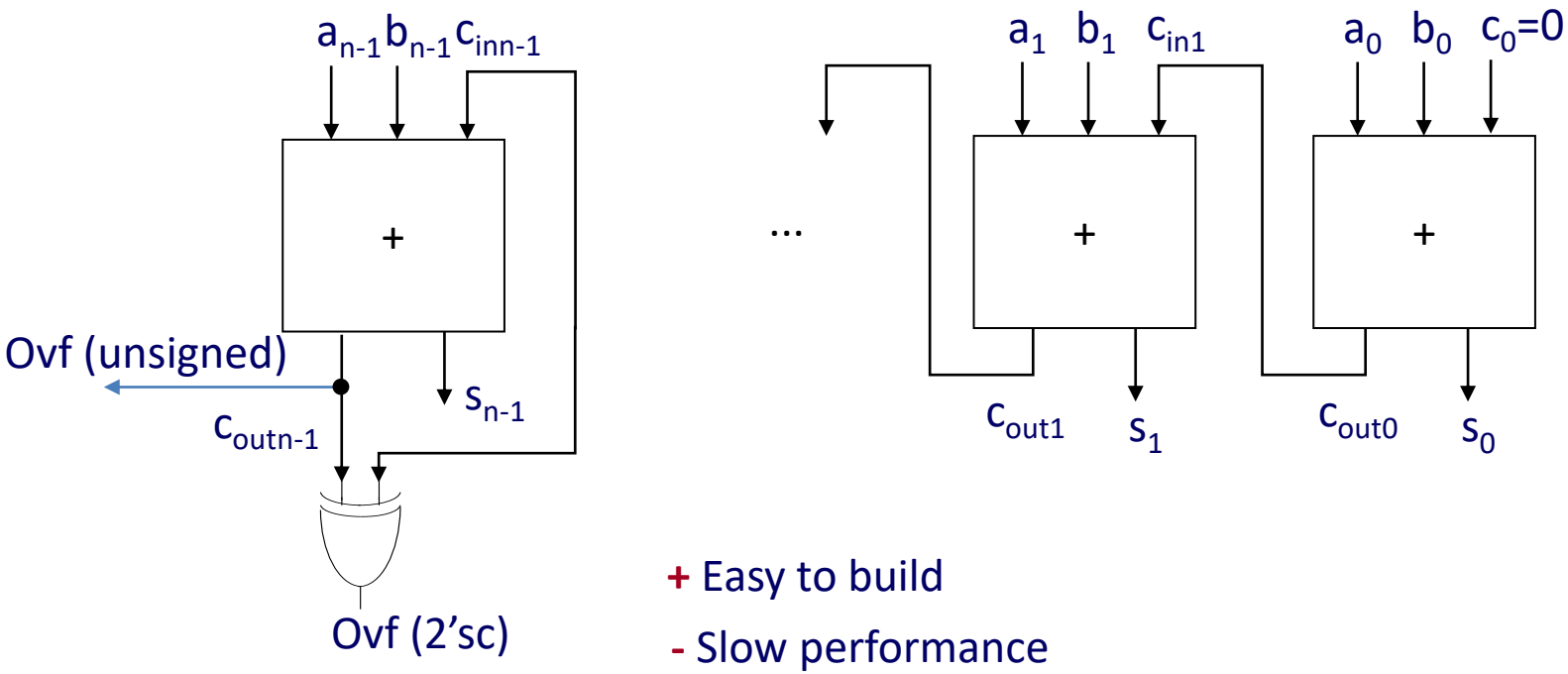
Exercise

- Show that the following circuit is Full-Adder



Ripple Adders

- The bit-wise addition and carry-propagation is implemented by a cascade of Full-Adders
- An iterative circuit paradigm



+ Easy to build

- Slow performance

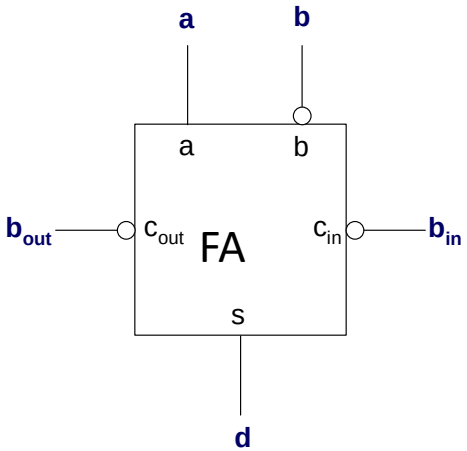
Full Subtractor

- Inputs: 2 single bit operands (a, b) and a Borrow-in bit (b_{in})
- Outputs
  - A 2 bit result:
    - The difference (d)
    - The Borrow-out (b_{out})
- Exercise

b_{in}	a	b	b_{out}	d
0	0	0	0	0
0	0	1	1	1
0	1	0	0	1
0	1	1	0	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	1	1

$$b_{out} = \overline{a} \cdot b + \overline{a} \cdot b_{in} + b \cdot b_{in}$$

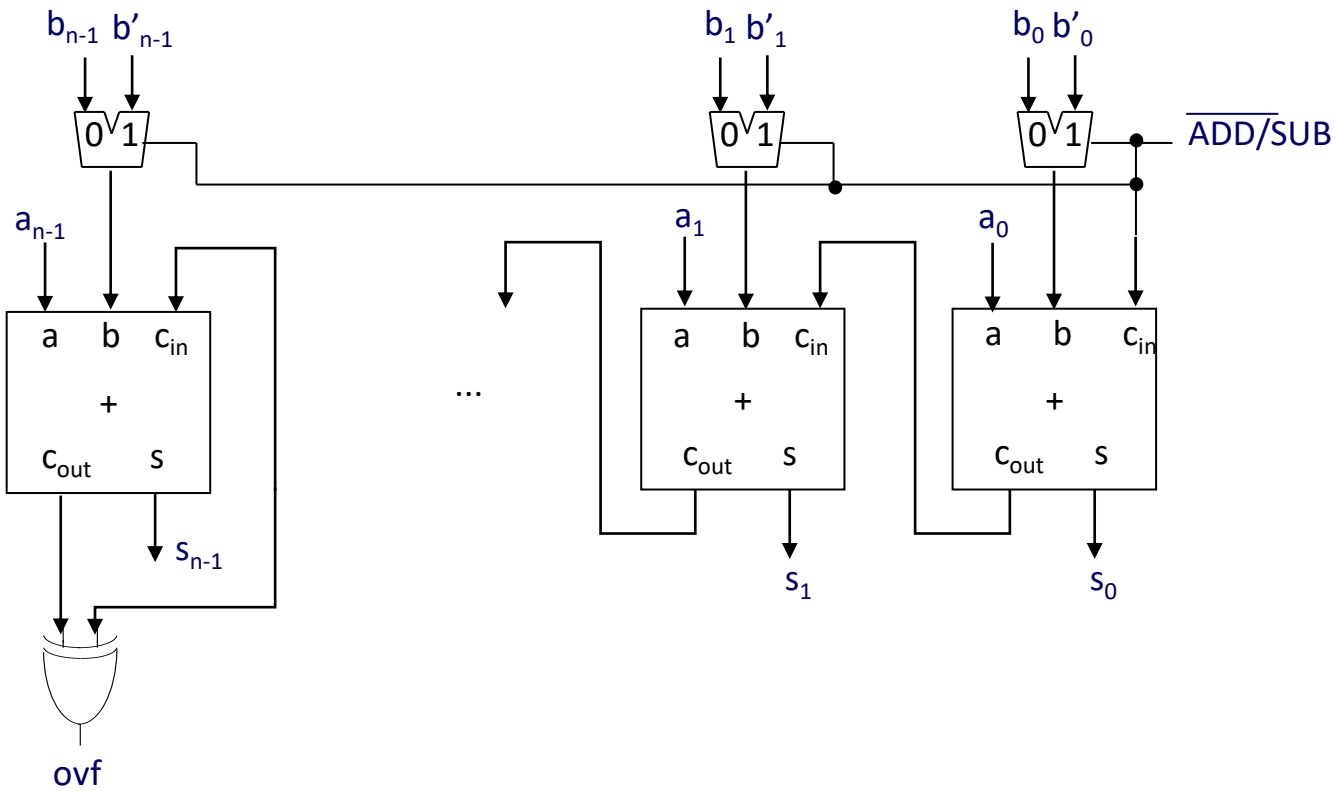
$$d = a \oplus b \oplus b_{in}$$



Write the output equations and show that the Full-Adder with modified inputs is a subtractor

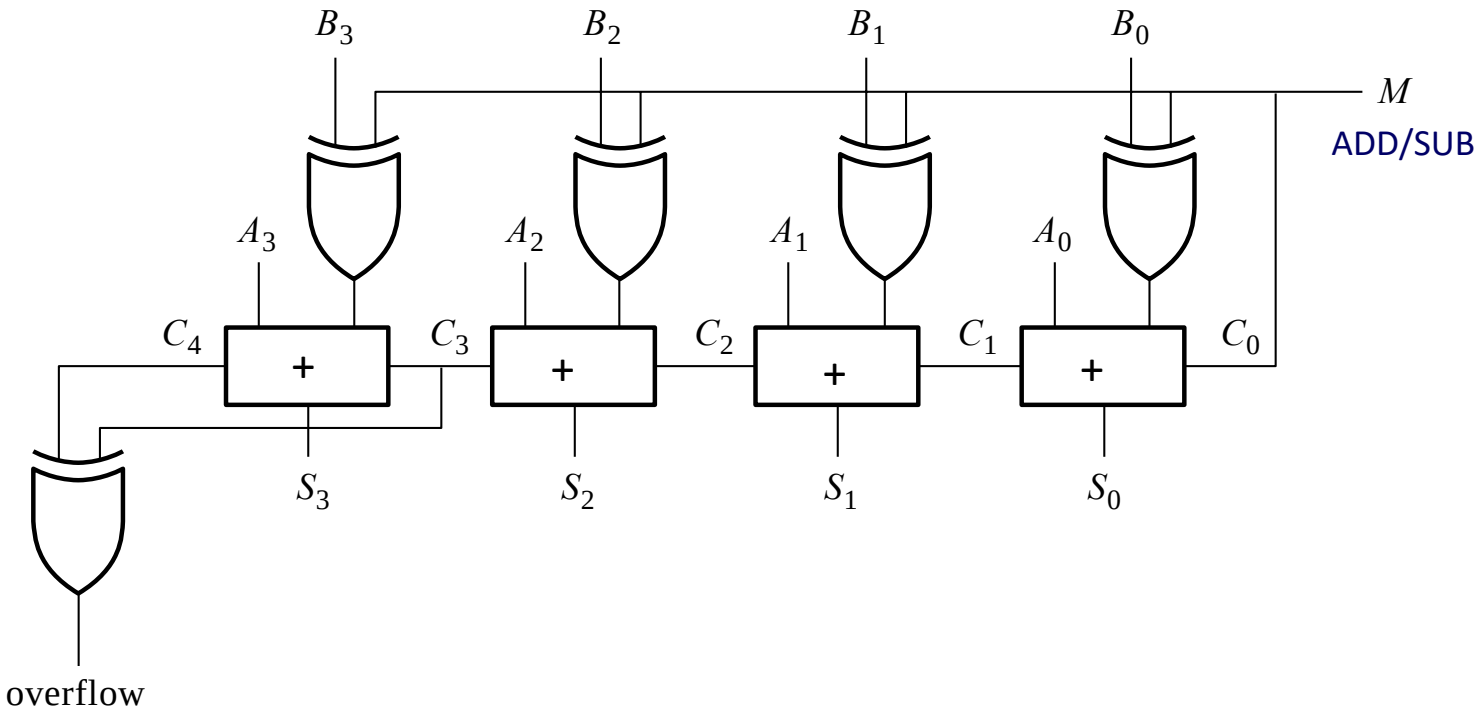
Ripple Adder/Subtractor

- Using 2's complement representation means the same hardware for addition and subtraction
  - "Muxed" b and b' inputs
  - Initial carry = 1 if subtraction

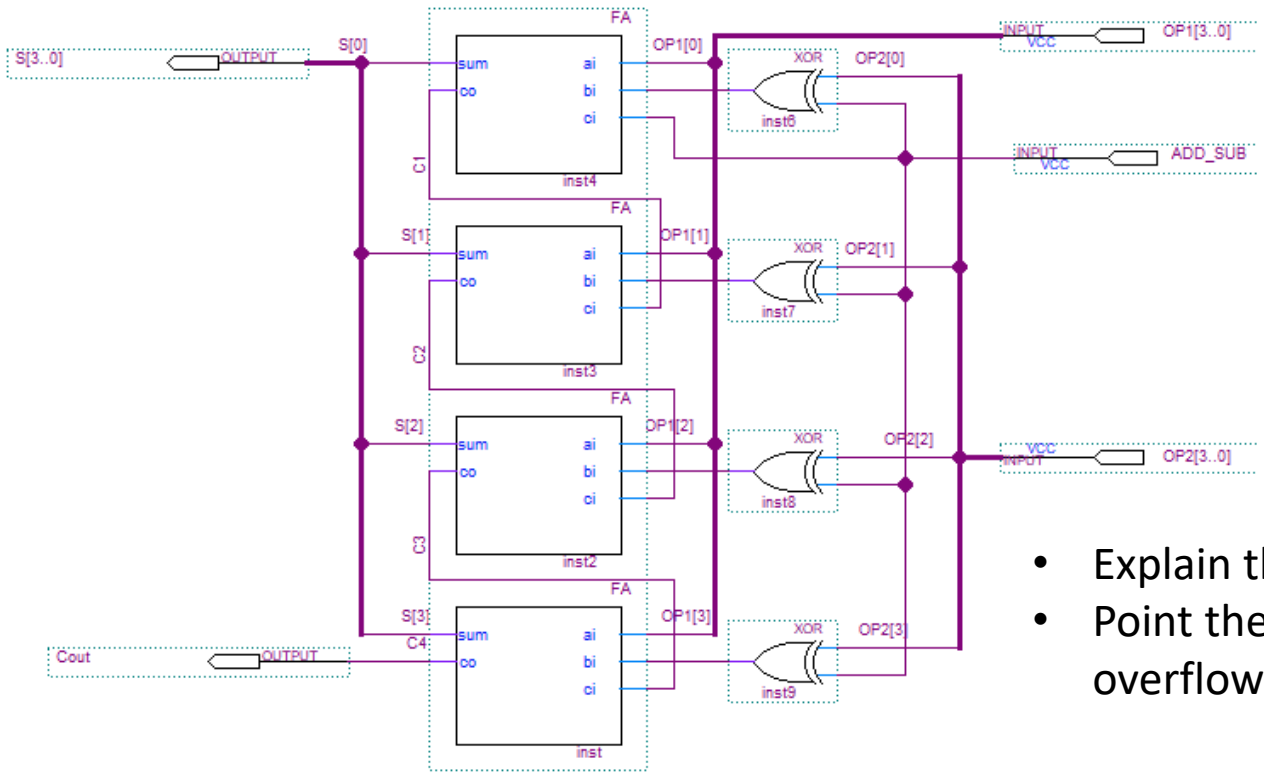


Exercise

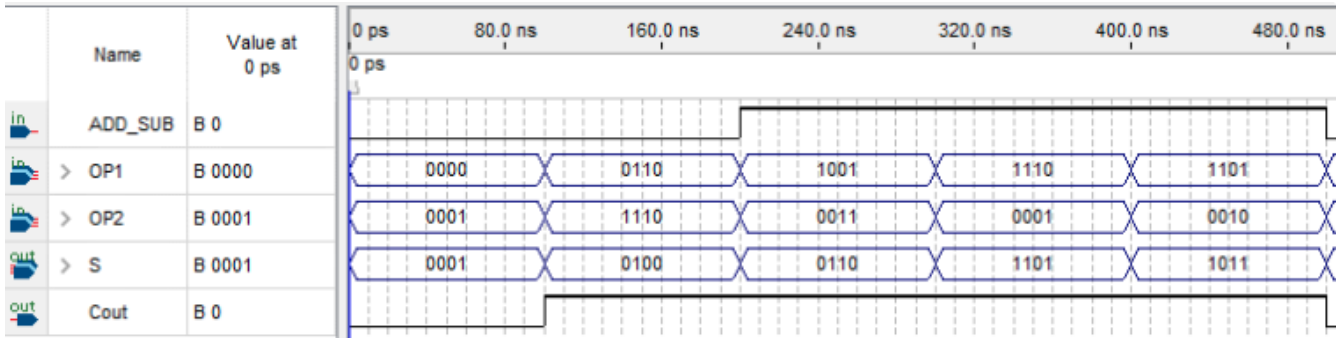
- Verify that the following implementation is equivalent to the adder/subtractor circuit of the previous slide



Exercise



- Explain the timing diagram.
- Point the time intervals where overflow occurs



Carry-Lookahead Adders (CLA)

- Ripple-Carry Adder (RCA)-Summary
- Structure:
  - built from several full adders connected in series.
- Operation:
  - each bit must wait for the carry from the previous bit before producing its result.
- Delay:
  - grows linearly with the number of bits ($O(n)$).
- Example:
  - in a 32-bit adder, the last bit is only computed after the carry "ripples" through all 31 earlier adders.
- Advantage:
  - simple and cheap in hardware.
- Disadvantage:
  - slow for large word sizes

Solution!
Carry-Lookahead Adders

Carry-Lookahead Adders (CLA)

- Carry-Lookahead Adder (CLA)
- Objective
  - avoid the accumulation of delays in the propagation of carry signals
- Structure:
  - uses extra logic to predict (lookahead) the carries in parallel, instead of waiting for them to ripple.
- Key concepts:
  - Generate (G): if a bit position always generates a carry.
  - Propagate (P): if a bit position only propagates the incoming carry.
  - Formulas allow all carries to be computed in parallel using logic gates.
- Delay:
  - grows logarithmically with the number of bits ($O(\log n)$), *much faster than RCA.*
- Advantage:
  - fast.
- Disadvantage:
  - requires more hardware and more complex logic.

Carry-Lookahead Adders (CLA)

- The idea: Compute the *ith* carry non-iteratively
- The starting points:
  - The usual equations

$$s_i = a_i \oplus b_i \oplus c_i$$

$$c_{i+1} = a_i \cdot b_i + a_i \cdot c_i + b_i \cdot c_i$$

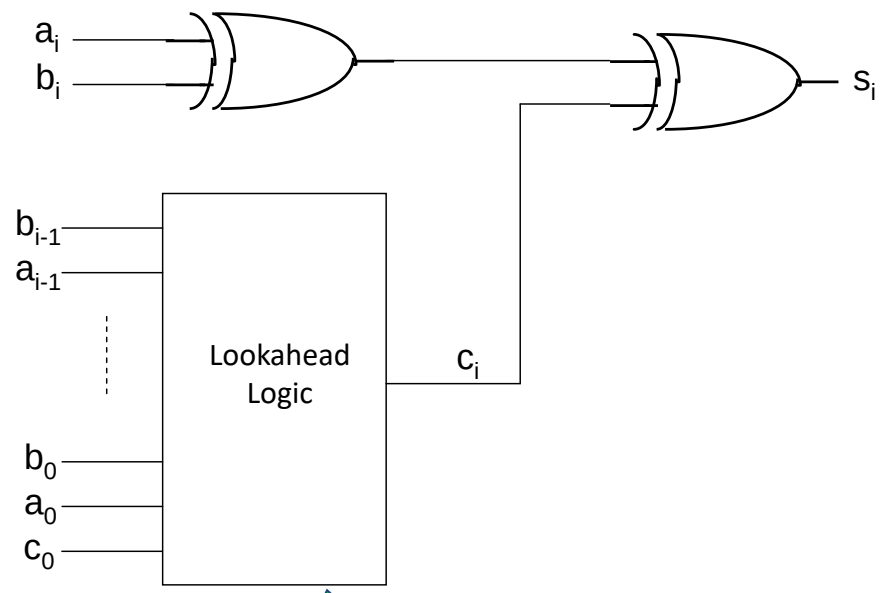
  - Intermediate functions
    - Carry Generation

$$g_i = a_i \cdot b_i$$

    - Carry Propagation

$$p_i = a_i + b_i$$

  - How to compute C_i?



Additional logic to generate information about previous carry amounts in each sum without resorting to previous sums.

Remarks about the Carry

- It is now necessary to determine when the C_i signal is generated. This signal is conditioned by the following definitions.
  - For a given combination of inputs a_i and b_i, the adder stage *i* generates a carry if it produces a carry output signal $C_{i+1} = 1$, regardless of the preceding inputs. We necessarily have a carry generation $C_{i+1} = 1$ whenever $a_i = b_i = 1$ ⇒ So $g_i = a_i.b_i$
  - For a given combination of inputs a_i and b_i, the adder stage *i* propagates a carry if it produces a carry output signal $C_{i+1} = 1$, in the presence of a combination of inputs that produces an input carry. If $a_i \neq b_i$ but $a_i+b_i = 1$ the carry propagates since $C_{i+1} = C_i$ ⇒ so $p_i = a_i + b_i$
- These equations state that a stage *generates* a carry if both bits to be added are 1 and *propagates* a carry if at least one of these input bits is 1.

Remarks about the Carry

- Finally, the Carry equation becomes

$$c_{i+1} = g_i + p_i \cdot c_i$$

- To eliminate carry propagation, it is now necessary to expand the carry terms for each floor.

$$c_1 = g_0 + p_0 \cdot c_0$$

$$c_2 = g_1 + p_1 \cdot c_1 = g_1 + p_1 \cdot (g_0 + p_0 \cdot c_0) = g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0$$

$$c_3 = g_2 + p_2 \cdot c_2 = g_2 + p_2 \cdot (g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0) =$$

$$= g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0$$

The 4 bit CLA

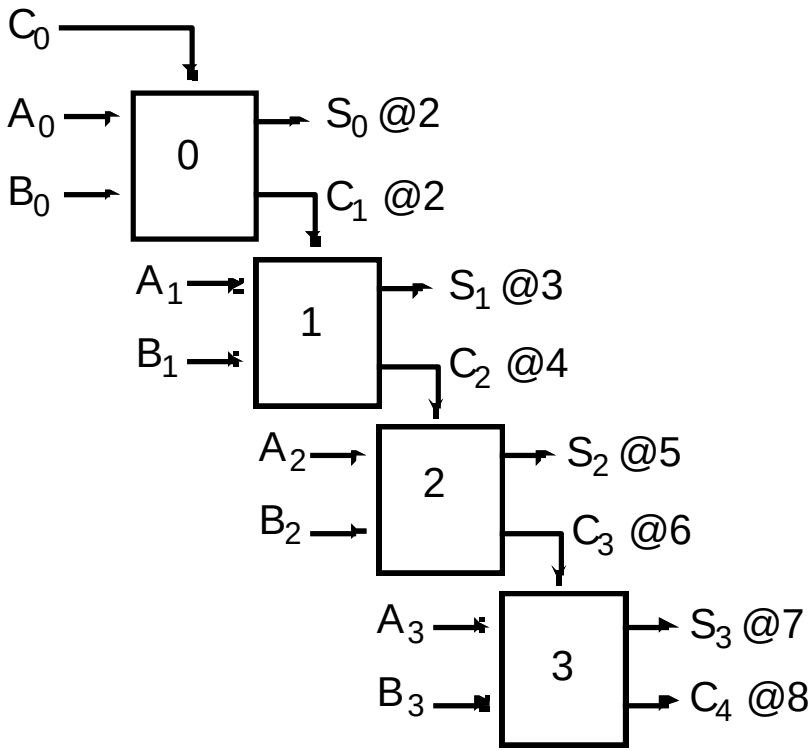
$$c_4 = g_3 + p_3 \cdot c_3 = g_3 + p_3 \cdot (g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0) =$$
$$= g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0 + p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0$$

- Each of these equations corresponds to a circuit with three delay levels, one to calculate the signals g_i and p_i and two to compute the the sum of the products.

- As can be seen from this 4-bit example, it is possible to avoid carry propagation, at the cost of additional logic that grows very rapidly with the number of bits used.

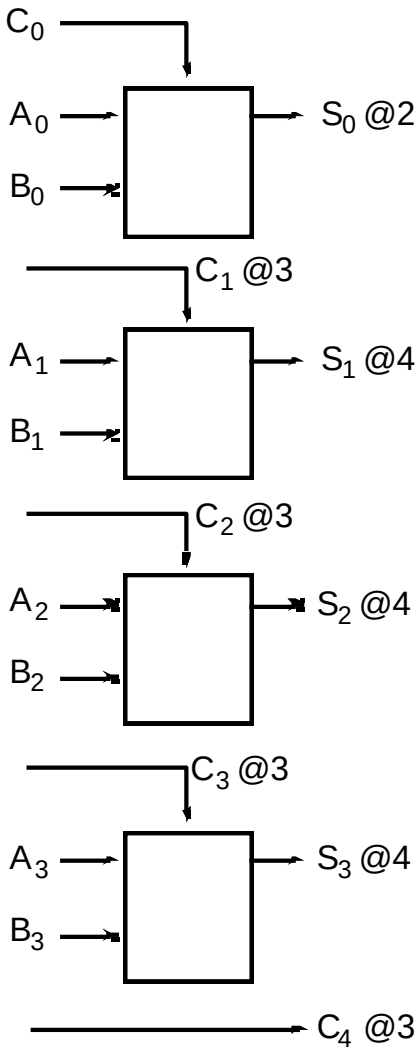
Ripple Adder versus CLA

- 4 bit ripple adder



Final result always after 2x4 = 8 delays

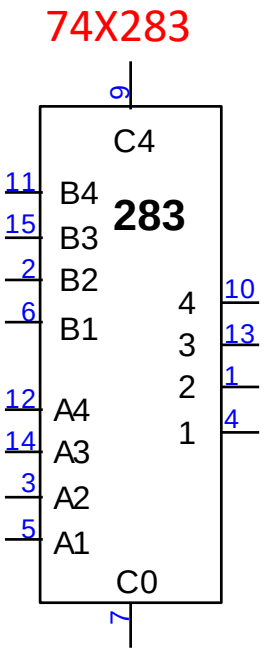
- 4 bit CLA

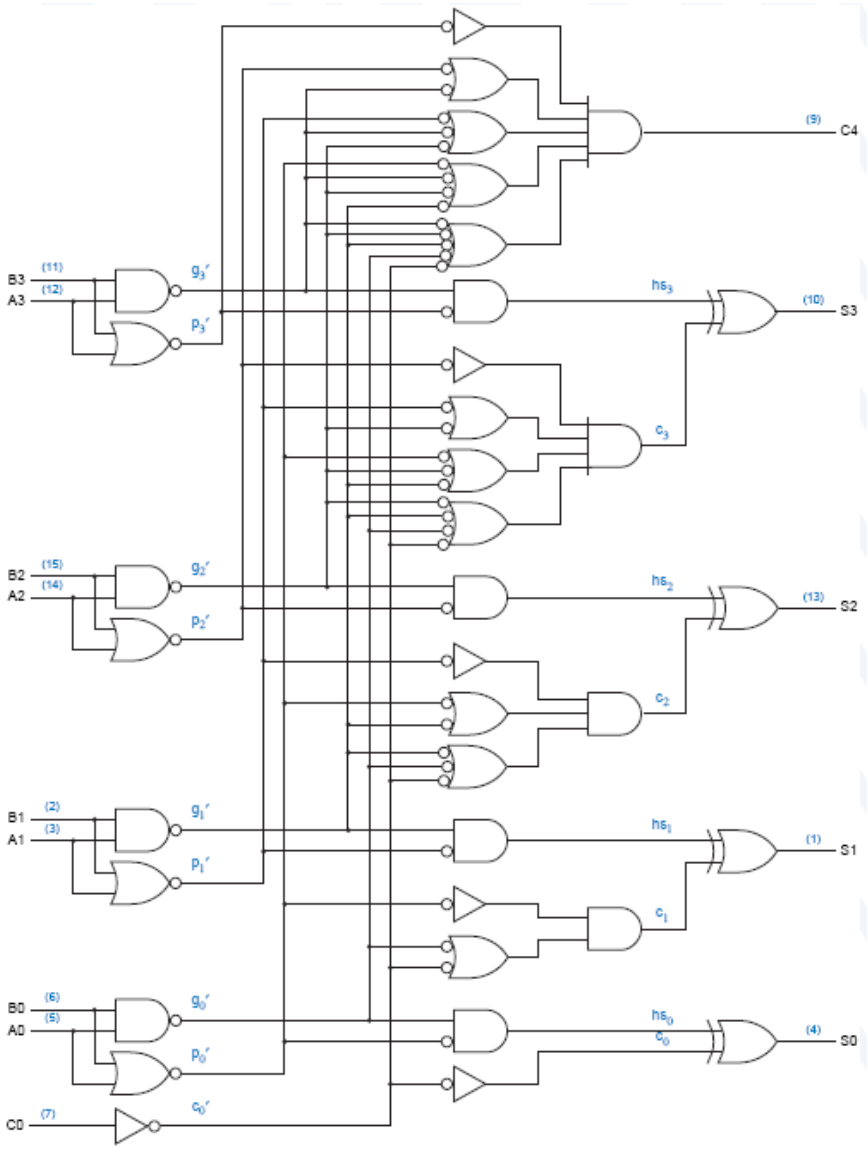


Final result always after 4 delays

The 74283 model

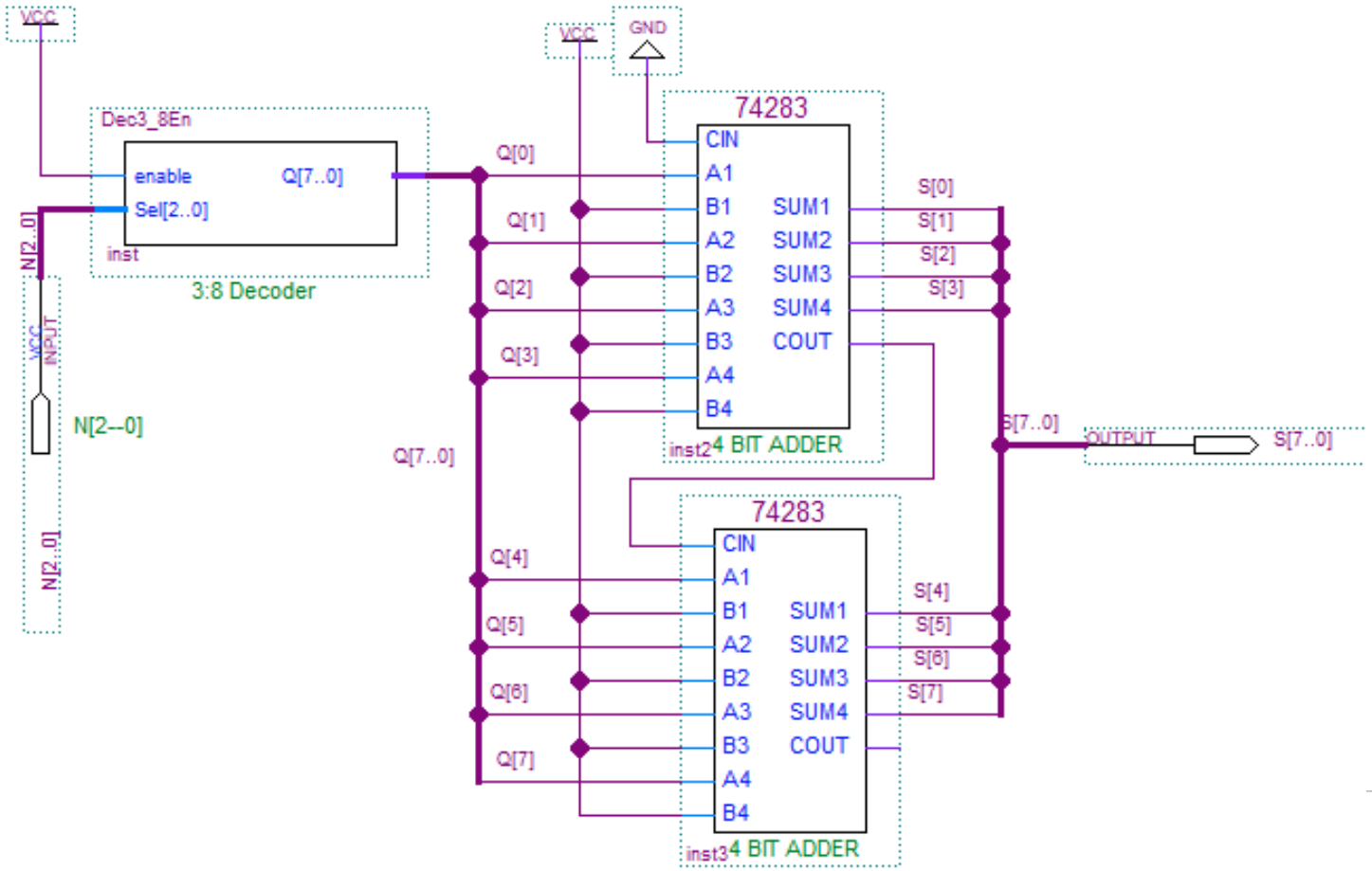
- CLA logic





Exercise

- In a 2's complement representation, what's the decimal value of the result S with N = 6_{10}



BCD Addition

- Possible results for 2 digit BCD addition with carry

soma	binário		BCD	
	carry out	soma	*carry out*	soma
0	0	0000	0	0000
1	0	0001	0	0001
2	0	0010	0	0010
3	0	0011	0	0011
4	0	0100	0	0100
5	0	0101	0	0101
6	0	0110	0	0110
7	0	0111	0	0111
8	0	1000	0	1000
9	0	1001	0	1001
10	0	1010	1	0000
11	0	1011	1	0001
12	0	1100	1	0010
13	0	1101	1	0011
14	0	1110	1	0100
15	0	1111	1	0101
16	1	0000	1	0110
17	1	0001	1	0111
18	1	0010	1	1000
19	1	0011	1	1001

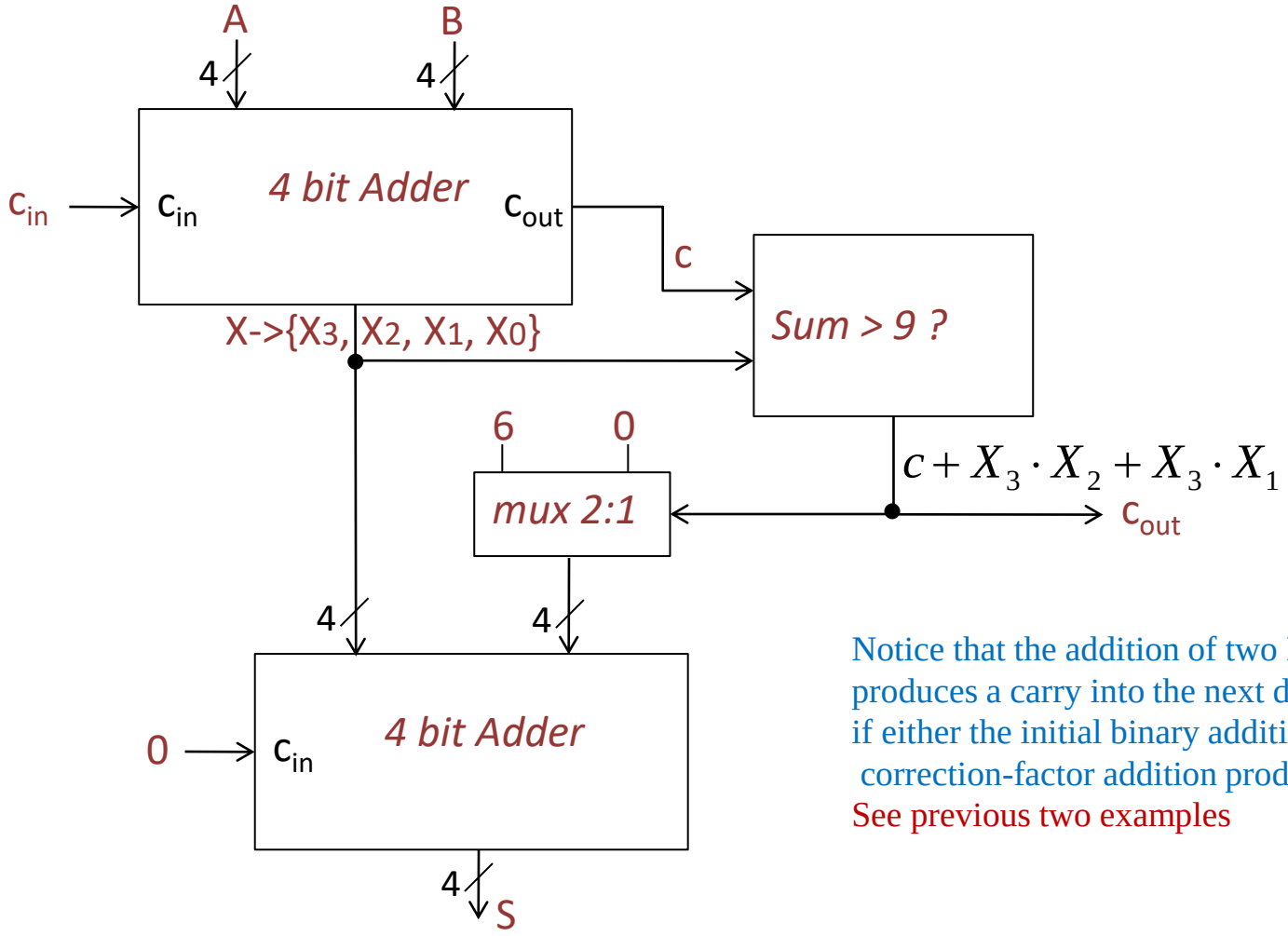
Offset correction required: add 6 to the binary result

Examples

```
      5      0101
  +   9    + 1001
  ------   ------
     14      1110
            + 0110  — correction
  ------   ------
  10 + 4    1 0100

      9      1001
  +   9    + 1001
  ------   ------
     18      1 0010
            + 0110  — correction
  ------   ------
  10 + 8    1 1000
```

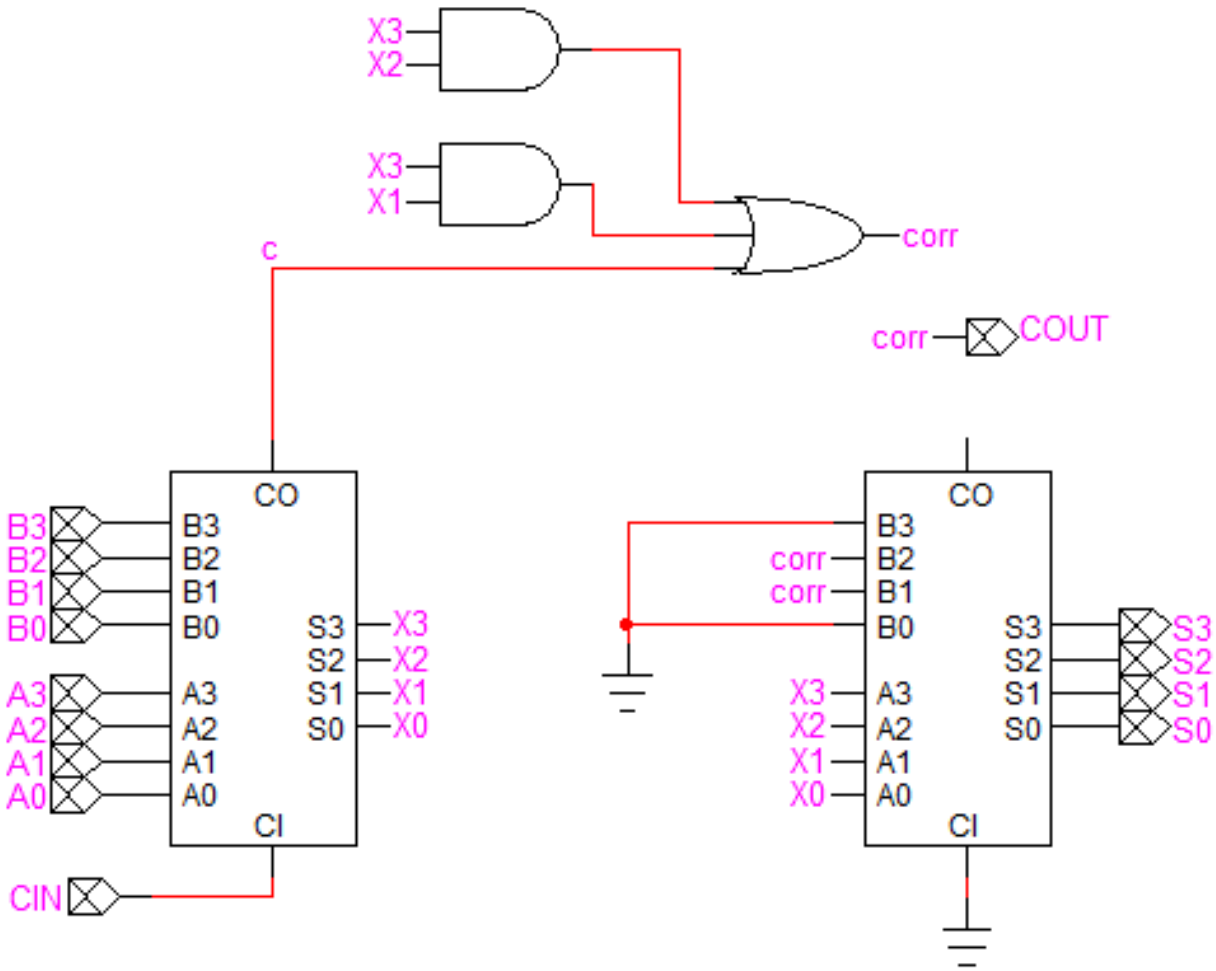
BCD addition algorithm



Notice that the addition of two BCD digits produces a carry into the next digit position if either the initial binary addition or the correction-factor addition produces a carry.

See previous two examples

Possible implementation



ALU

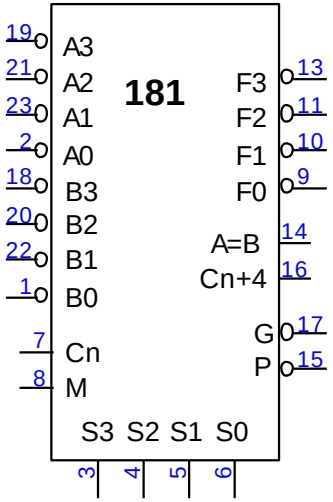
- An arithmetic logic unit is a combinational block that executes any logical or arithmetic operation over a pair of *b* bits operands.

- There is a mode input that chooses between the logical or arithmetic behavior

- There is a op-code set of inputs that choose a particular operation from a limited “operation” set.

The 74181 ALU

- Example



S3	S2	S1	S0	M=0 (op. aritm.)	M=1 (op. lógica)
0	0	0	0	F = A – 1 + CIN	F = $\overline{A}$
0	0	0	1	F = A *AND* B – 1 + CIN	F = $\overline{A}$ *OR* $\overline{B}$
0	0	1	0	F = A *AND* $\overline{B}$ – 1 + CIN	F = $\overline{A}$ *OR* B
0	0	1	1	F = 1111 + CIN	F = 1111
0	1	0	0	F = A + (A *OR* $\overline{B}$) + CIN	F = $\overline{A}$ *AND* $\overline{B}$
0	1	0	1	F = A *AND* B + (A *OR* $\overline{B}$) + CIN	F = $\overline{B}$
0	1	1	0	F = A – B – 1 + CIN	F = A *XOR* $\overline{B}$
0	1	1	1	F = A *OR* $\overline{B}$ + CIN	F = A *OR* $\overline{B}$
1	0	0	0	F = A + (A *OR* B) + CIN	F = $\overline{A}$ *AND* B
1	0	0	1	F = A + B + CIN	F = A *XOR* B
1	0	1	0	F = A *AND* $\overline{B}$ + (A *OR* B) + CIN	F = B
1	0	1	1	F = A *OR* B + CIN	F = A *OR* B
1	1	0	0	F = A + A + CIN	F = 0000
1	1	0	1	F = A *AND* B + A + CIN	F = A *AND* $\overline{B}$
1	1	1	0	F = A *AND* $\overline{B}$ + A + CIN	F = A *AND* B
1	1	1	1	F = A + CIN	F = A

Unsigned Multiplication

- We follow the same rules of the decimal system

$$12 \times 13 = 156$$

				1	1	0	0	← multiplicand
			×	1	1	0	1	← multiplier
				1	1	0	0	Partial products
			0	0	0	0		
		1	1	0	0			
	1	1	0	0				
1	0	0	1	1	1	0	0	Final product

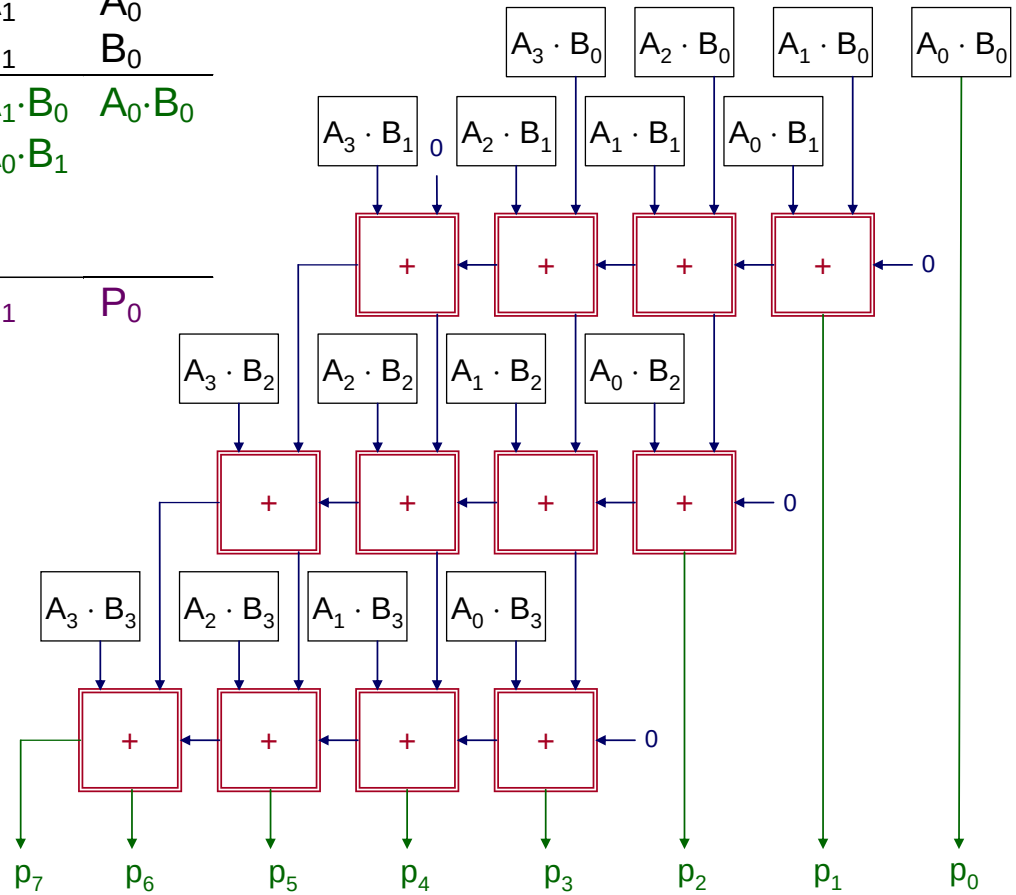
Combinational Multipliers

				A_3	A_2	A_1	A_0
			$\times$	B_3	B_2	B_1	B_0
				$A_3 \cdot B_0$	$A_2 \cdot B_0$	$A_1 \cdot B_0$	$A_0 \cdot B_0$
			$A_3 \cdot B_1$	$A_2 \cdot B_1$	$A_1 \cdot B_1$	$A_0 \cdot B_1$	
		$A_3 \cdot B_2$	$A_2 \cdot B_2$	$A_1 \cdot B_2$	$A_0 \cdot B_2$		
	$A_3 \cdot B_3$	$A_2 \cdot B_3$	$A_1 \cdot B_3$	$A_0 \cdot B_3$			
P_7	P_6	P_5	P_4	P_3	P_2	P_1	P_0

				1	1	0	0
			$\times$	1	1	0	1
				1	1	0	0
			0	0	0	0	
		1	1	0	0		
	1	1	0	0			
1	0	0	1	1	1	0	0

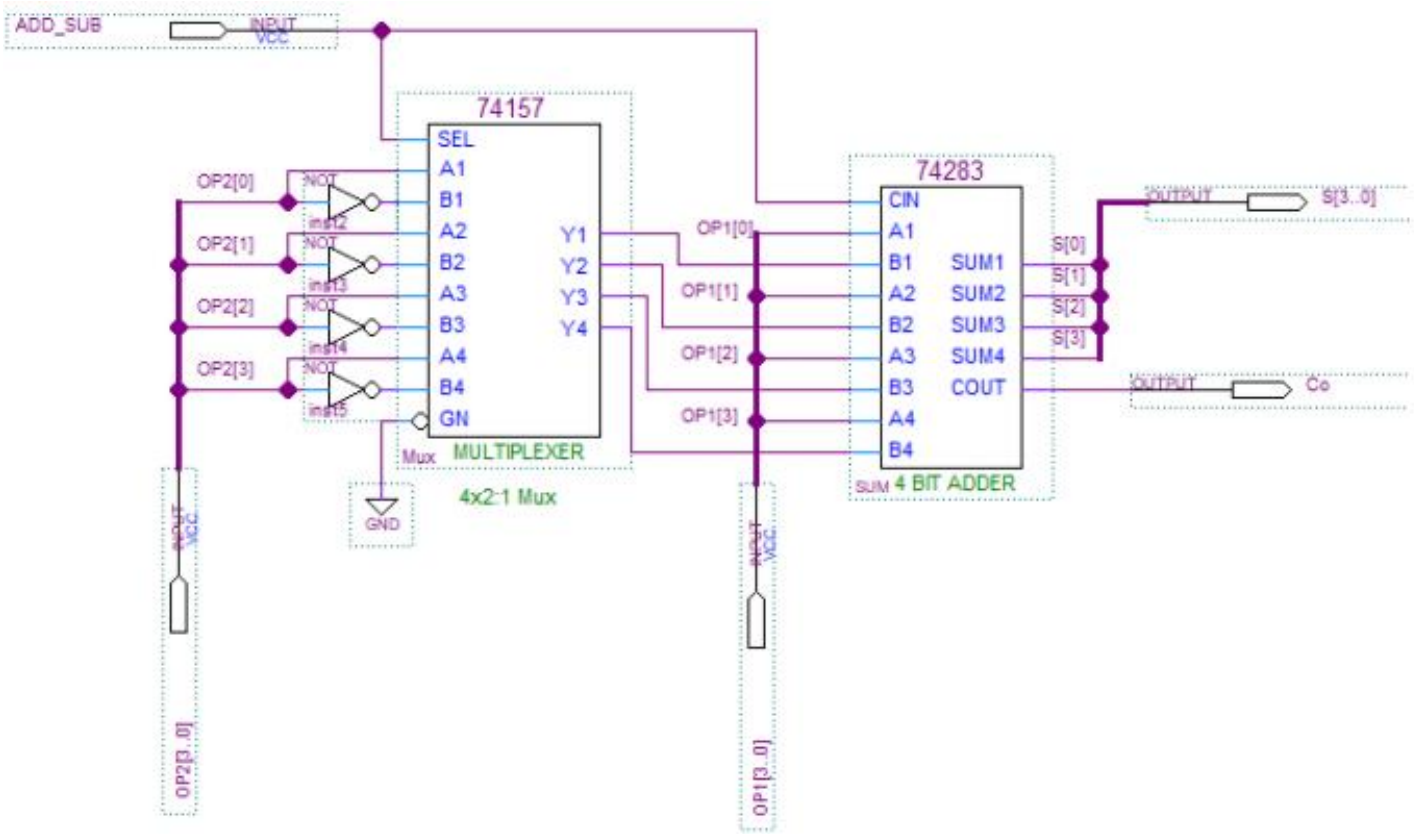
16 × AND-2

12 adders



Exercise

- Analyze the following circuit and determine the result for OP1 = 1111, OP2 = 0111 and ADD_SUB=1



Final Remarks

- Always recall
  - The block symbol
  - The types of inputs (operands) and outputs
  - Distinguish between iterative and non-iterative solutions

- Design with encapsulated logic requires mastering all the functional details of each block